Round 2 - Prototype Development

Congratulations on advancing to Round 2. You've already pitched an initial concept for one of the four problem statements — now it's time to go deeper. This document expands your chosen track with the real-world complexity that companies grapple with, along with a broader option for solutioning areas you might explore. It is still intentionally open: there is no single "correct" architecture, and you are not expected to have access to a real company's proprietary data. Use reasonable assumptions and focus on innovation, creativity, and technical novelty.

Each problem track below expands on its Round 1 brief with deeper context, the real complexities such problems present inside real organizations, and a broad set of solutioning areas you're free to draw from selectively — you are not expected to address every point listed.

PROBLEM TRACK 1

ControlPlane.ai

Recap & Expanded Context

In Round 1, you explored the concept of a Responsible AI Checker — a layer that evaluates AI responses in real time and flags or blocks bias, hallucination risk, or privacy leaks before they reach a user. In practice, enterprises run generative AI across s many different use cases at once — customer-facing chatbots, internal copilots for employees, decision-support tools embedded in regulated workflows — and each of these carries a different risk signature depending on the model, the data it draws on, and how its output is used downstream.

For Round 2, you'll take this concept further: design a more complete solution and build a working prototype that demonstrates its core mechanism, even on a limited or simulated scope.

Real-World Complexities to Consider

- Different AI use cases (customer-facing vs. internal, real-time vs. batch) have very different risk tolerance and latency budgets — a single, one-size-fits-all checking approach rarely works well everywhere.

- Bias, hallucination, and privacy risks often overlap in practice — a fabricated detail about a person can simultaneously be a hallucination and a privacy concern — making clean categorization harder than it first appears.
- There is often no reliable, real-time "ground truth" to check a claim against — the same knowledge gaps that cause hallucination can make automated verification difficult too.
- Over-flagging creates alert fatigue and pushes users to ignore or bypass warnings; under-flagging creates real liability — most real systems have to deliberately tune this tradeoff rather than solve it away.
- Multi-turn conversations and AI agents that take actions (not just generate text) introduce compounding risk, where one questionable output can shape several downstream decisions.
- Regulatory expectations differ by geography and industry (e.g., data protection law, emerging AI-specific regulation, sector rules) and continue to evolve, so rigid, hard-coded rules age quickly.
- Enterprises typically consume a foundation model via API rather than owning it outright, limiting how deeply a checker can inspect model internals versus working at the input/output layer.

Solutioning Areas You Could Explore

- Detection techniques — rule-based heuristics, embedding/statistical anomaly detection, a secondary "AI-as-judge" pattern, retrieval verification against source documents, dedicated PII/entity detection
- Decision logic — confidence scoring, tiered responses (allow / edit / flag for review / block), and clear rules for when a human should be pulled in
- Architecture — where the checker sits in the pipeline (pre-response gate, inline middleware, post-hoc audit), and how checks can run in parallel to protect latency
- Governance — a configurable policy layer so behavior can vary by use case, geography, or risk appetite, with a clear audit trail behind every decision
- Feedback loops — how flagged or overridden cases feedback to improve detection quality over time
- Metrics & monitoring — how you would define, measure, and report false positive/negative rates and overall system trustworthiness to a skeptical stakeholder

Reference Parameters (Illustrative — Adapt Freely)

- Assume an enterprise operating multiple AI use cases at once (for example, a customer support assistant, an internal knowledge assistant, and a decision-support tool), each with different latency and risk tolerance
- Assume tens of thousands of interactions per week across these use cases combined
- Assume a mix of well-governed and loosely governed internal data sources feeding these AI systems

These parameters are directional, not a fixed dataset — you're encouraged to make your own reasonable assumptions, state them clearly, and design a solution that would generalize for broader adoption.

PROBLEM TRACK 2

PatientTriage.ai

Recap & Expanded Context

In Round 1, you explored a patient triage assistant that helps hospital staff prioritize and route patients as they arrive, aiming to reduce wait times without replacing clinical judgment. In practice, no two

emergency departments look alike patient mix, staffing levels, and existing systems vary enormously, and the tool must work under real time pressure with imperfect information.

For Round 2, develop your concept into a more complete solution, backed by a working prototype that demonstrates how it would behave with realistic (even if simplified or simulated) patient data.

Real-World Complexities to Consider

- Patients present with overlapping or ambiguous symptoms that don't map cleanly onto standard severity scales — some patients under-report pain or symptoms, and presentation can differ significantly by age or condition.
- Vital sign thresholds and symptom weights differ significantly across pediatric, adult, and geriatric populations — a fever of 38.5°C carries different clinical urgency in a 3-year-old versus a 75-year-old. Solutions that apply to a single adult-calibrated scoring model across all age groups introduce silent safety risk.
- Data quality and availability at intake varies hugely — a returning patient may have a rich history in the hospital's systems, while a first-time patient may have almost nothing beyond what's observed in the moment.
- Triage decisions must be made — and be explainable — within seconds, by a clinician who is often simultaneously managing several other patients.
- Under-triage and over-triage carry asymmetric costs — missing a critical case is categorically worse than over-prioritizing a minor one. Any solution must be deliberately tuned to bias toward escalation under uncertainty rather than optimized for average accuracy, and teams must demonstrate this design choice explicitly in their prototype.
- Hospitals differ enormously in scale, specialty mix, and staffing — a workflow that works for a large urban trauma center may not transfer to a small rural emergency department.
- Clinical accountability and liability mean any recommendation must remain reviewable and overridable by a licensed clinician, with a clear audit trail and compliance with health-data regulation.
- Integration with existing hospital systems (patient records, bed management, staff rosters) is rarely simple, and system maturity varies a great deal from one hospital to the next.

Solutioning Areas You Could Explore

- Data strategy — how you'd structure and weigh available inputs (vitals, self-reported symptoms, history, observed cues) despite inconsistent completeness
- Decision model — rules-based scoring, ML-based risk scoring, or a hybrid, and how you would represent the assistant's own uncertainty
- Workflow design — how a recommendation is surfaced to a nurse in the moment, how overrides are captured, and how the system behaves differently during a surge versus a quiet shift
- Safety-first design — sensible fail-safe defaults (for example, escalating rather than downgrading when uncertain), and ongoing monitoring of waiting patients for signs of deterioration. The system must monitor patients already in the waiting queue and trigger re-assessment if wait time exceeds safe thresholds for their severity level or if vitals are re-recorded as worsening.
- Adoption & change management — how you'd get a fatigued, time-pressured staff to actually trust and use the tool rather than work around it
- Patient data protection – how would the patient data be protected from unfair and unathorised usage.
- Scalability — how the same underlying assistant could flex across hospitals of very different size, specialty mix, and technical maturity

Reference Parameters (Illustrative — Adapt Freely)

- Assume emergency departments ranging from roughly 100 to 500+ patient visits per day
- You may reference standard triage frameworks (e.g., a 5-level severity scale) or propose an alternative
- Assume mixed data availability — roughly half of arriving patients have some prior health record on file, half do not
- State your assumed regulatory jurisdiction (e.g., HIPAA in the US, GDPR + national health law in the EU, or a named equivalent). This affects your audit trail design, data retention policy, consent model, and what a clinician override must legally record.

These parameters are directional, not a fixed dataset — you're encouraged to make your own reasonable assumptions, state them clearly, and design a solution that would generalize for broader adoption.

Minimum Prototype Expectations (Illustrative):

- Demonstrate triage scoring on at least 15–20 simulated patient records
- Include at least one ambiguous presentation, one pediatric/geriatric case, and one zero-history (first-time) patient
- Show how the system behaves under a simulated surge (e.g., 3× normal volume)
- Surface uncertainty explicitly — the prototype must not return a score without a confidence indicator
- Capture at least one clinician override and show what the system logs

PROBLEM TRACK 3

BusinessIntelligence.ai

Recap & Expanded Context

In Round 1, you explored a KPI storytelling engine that explains what changed in a business metric, identifies likely root causes, and recommends next steps in plain language. In practice, most businesses track KPIs across fragmented systems with different refresh cadences and granularities, and the "right" explanation for a movement often depends on who's asking and what they plan to do about it.

Round 2 Objective

Design and demonstrate a working prototype of a **KPI intelligence-to-action engine** that:

1. Detects and prioritises material KPI movements.
2. Reconciles data and business context across heterogeneous sources.
3. Identifies and ranks explanatory drivers using appropriate analytical methods.
4. Generates persona-specific narratives supported by traceable evidence.
5. Communicates uncertainty and abstains when evidence is insufficient or contradictory.
6. Recommends practical actions grounded in business levers, constraints and decision rights.
7. Mechanism to learns from analyst and business-user feedback.

8. Operates within realistic security, cost, latency and scalability constraints.

The LLM should not be treated as **the source of quantitative truth**. Teams should explicitly demonstrate when they use **deterministic logic, SQL, business rules, statistics, traditional ML, causal inference, retrieval or LLMs—and why**.

Real-World Complexities to Consider

- Multiple interacting drivers such as price, volume, mix, marketing, supply, seasonality, competition and external events.
- Different source-system refresh cadences, grains, data quality levels and historical coverage.
- Inconsistent KPI definitions, hierarchies, calendars, business rules and aggregation logic.
- Sparse history for new products, categories or markets.
- Materiality based on both statistical significance and business impact.
- Contradictory evidence, missing data and confidence calibration.
- Role-based personalization of insight depth, recommended actions and delivery channels.
- Row-, column- and domain-level security, sensitive-data protection and auditability.
- Model and data drift, feedback capture and continuous evaluation.
- LLM economics, including model choice, token consumption, latency, caching and cost per insight.

Solutioning Areas You Could Explore

Teams may explore a hybrid combination of:

- Anomaly detection, contribution analysis, forecasting, causal inference and business-rule reasoning.
- Governed KPI semantics, metadata, lineage, business rules, ontology or knowledge graphs.
- LLM-assisted intent understanding, orchestration, narrative synthesis and contextual retrieval.
- Proactive alerts, conversational analysis, augmented dashboards or decision workspaces.
- Confidence scoring, evidence citation, alternative hypotheses and abstention mechanisms.
- Action recommendations structured as: **driver → controllable lever → action → expected impact → owner → confidence → monitoring plan**
- Human feedback, expert validation, correction workflows and learning loops.
- Platform-native and custom capabilities using Databricks, Snowflake, Microsoft Fabric, Tableau, Qlik, Looker or another suitable technology. (**Open to chose** any platform, or build completely custom solution or hybrid)

Platform-specific solutions are acceptable, but teams should distinguish between **native, configured, custom-built and externally integrated capabilities.**

Minimum Prototype Expectations:

- Three to five connected KPIs across two or three data sources with different grains or refresh cadences.
- A lightweight KPI or semantic contract covering definitions, calculations, drivers, thresholds, lineage and access restrictions.
- At least two personas receiving different insight narratives or recommended actions.
- One multi-factor KPI movement with known or simulated underlying drivers.

- One low-confidence scenario in which the engine requests clarification or abstains.
- One sparse-history or newly launched KPI scenario.
- One role-based security or entitlement scenario.
- Evidence showing source freshness, analytical method, contribution, confidence and lineage.
- A clear breakdown of LLM versus non-LLM processing.
- Runtime telemetry covering latency, model calls, token usage and estimated cost.

PROBLEM TRACK 4

DigitalTwin.ai

Recap & Expanded Context

In Round 1, you explored a digital twin prototype for a vehicle assembly line that helps plant teams see where bottlenecks are forming and predict likely defects before they happen. In practice, most assembly lines are a patchwork of legacy and modern equipment, and any twin must work with uneven sensor coverage rather than a perfectly instrumented, ideal factory.

For Round 2, extend your concept into a fuller solution design, backed by a working prototype that demonstrates the core predictive mechanism on realistic (even if simplified or simulated) production data.

Real-World Complexities to Consider

- Assembly lines mix legacy and modern equipment, so sensor coverage is often inconsistent — some stations are richly instrumented, others rely entirely on manual checklists.
- Bottlenecks and defects often have multi-causal, intermittent root causes (equipment wear, operator variation, upstream part quality, environmental conditions) that are hard to isolate from data alone.
- Modifying live production systems (PLCs, line control logic) carries real operational risk, and most plants only allow retrofits during scheduled, infrequent maintenance windows.
- A defect introduced early in the line may not surface until a much later inspection point, by which time many downstream units may carry the same undetected issue — making root-cause tracing after the fact especially difficult.
- Different stakeholders need very different views of the same twin — a floor supervisor needs real-time, in-the-moment signals, a plant manager needs weekly planning trends, and leadership needs a rollout business case.
- Extending a solution beyond a single line or plant means accounting for real variation in layout, equipment vintage, and sensor maturity across different sites.
- Predictive claims must be validated against real outcomes over time — false alarms about defects that don't materialise can erode floor-level trust in the system quickly.

Solutioning Areas You Could Explore

- Modelling approach — what to represent explicitly (cycle time, torque, vibration, temperature, throughput) versus infer indirectly, especially at sensor-poor stations
- Predictive techniques — anomaly detection, statistical process control, physics-informed models, or ML-based bottleneck/defect prediction, and how you'd validate them before trusting their output
- Handling data gaps — how the twin stays useful at stations with partial or no instrumentation, including any low-cost sensing you might propose

- User experience — distinct views for a floor supervisor's real-time needs, a plant manager's planning needs, and leadership's investment case, from the same underlying model
- Integration approach — working around legacy PLCs, OT data and live-production constraints without disrupting ongoing operations
- Scalability & ROI — how a prototype built for one line could reasonably extend to other lines, plants, or sites with different starting conditions

Reference Parameters (Illustrative — Adapt Freely)

- Assume a mixed-model assembly line with roughly 30–50 stations across body construction, paint, and final assembly
- Assume meaningful but uneven sensor coverage — a majority of stations well-instrumented, a meaningful minority reliant on manual checks
- Assume production can only be paused for instrumentation changes during a small number of scheduled maintenance windows per year

These parameters are directional, not a fixed dataset — you're encouraged to make your own reasonable assumptions, state them clearly, and design a solution that would generalize beyond one specific company.

What Round 2 Asks You to Deliver

- Detailed Business Proposal — problem framing, solution design, target users, business case and impact, a phased roadmap, and key risks with mitigations
- Working Prototype — a functional demonstration of your solution's core mechanism. It does not need to be production-grade or use real enterprise data; a working proof-of-concept on illustrative or sample data is expected and encouraged
- Public GitHub repository, including a prototype demo video and a README document